

SKIjack: A High-Level Native Language for the *SKI* Calculus

N. E. Davis

September 18, 2026

Abstract

Over a homoiconic calculus the designer chooses where quotation happens. Over a calculus without self-quotation the choice is not available: no combinator can take another apart, so reification is metalevel by theorem, and a language’s staging boundary is fixed before its syntax is. We present SKIjack, a language for writing metacircular interpreters over the *SKI* calculus, whose feature set is read off an existing artifact rather than designed: the 618-atom self-interpreter of [Davis](#), decomposed into its fuel loop, its dispatch, its spine walker and its atoms, with each part promoted to a surface construct. Declaring an object language generates that furniture, so authoring the semantics of a new symbol — the operation the first paper performed by hand — becomes a two-line edit. Quotation is available at compile time only, and the checker refuses statically the positions at which a function would be asked to behave as data. A scry namespace is a language feature, with paths as a declared type and blocking sound under an append-only namespace. The abstraction is free, and we show it by equality of terms: six programs compile byte-identically to the hand-built artifacts, including one in which nothing is generated, and reproduce their contraction counts.

1 Introduction

Over a homoiconic calculus (Lisp, Nock) the metacircular tradition gets quotation for free and the language designer chooses where evaluation stages live. Over a non-homoiconic calculus that choice is taken away: Proposition 3.1 says no term can quote a live term, so the compile-time/runtime line is fixed before the language is designed, and the two operations the calculus cannot do for itself, quote and recognize, go to the compiler and the virtual machine respectively. [Davis](#) presented a metacircular evaluator for *SKI* which is written and evaluated in the same combinator system. This capability immediately suggests the construction of a battery of computational conveniences and abstractions within the same system; to wit, a language. We present SKIjack (pronounced “skyjack”), a closed-supercombinator macro-like language built in and targeting the non-homoiconic *SKI* calculus as its instruction set architecture. Every named thing compiles to a closed term and every name is resolved before anything runs, so the language has no runtime environment to address: bracket abstraction has already closed what a subject-oriented calculus would have to carry. SKIjack is evaluable on any *SKI* interpreter as a virtual machine.

The asymmetry that forces the staging line is older than either calculus’s programming languages, and naming it early makes the rest of the design read as consequence rather than taste. [Böhm \[1968\]](#) proved that two distinct $\beta\eta$ -normal forms can always be told apart from

the outside: some context applies them to arguments and recovers any two results we please.¹ Jay and Given-Wilson [2011] proved that no combinator can take another apart from the inside. Both papers of this pair live in the gap between those two results. Reading a compiled term by applying it to fresh markers and seeing which one comes back is the outside; Proposition 3.1 is the inside; and a VM jet is permitted to factorise a live term precisely because it is the Turing machine with the term on its tape, which is to say outside as well. Quotation goes to the compiler and recognition to the VM not because that is convenient but because those are the only places the operations exist.

The **whnfF** metacircular interpreter and **UQ** unquoter function presented in Davis are the starting point for our work. That paper discussed extensions to the *SKI* calculus built on higher layers of interpreters, introducing an **Err** error type, a **Maybe** partiality type, and discussed implications such as namespace value retrieval and other innovations. By authoring a new symbol at one site, the **whnfF** approach enables extension and modification of the combinator’s semantics, permitting it to behave more like a language than a mere combinator system. This paper presents SKIjack and three novel contributions for *SKI* combinator research: a two-level execution model with a kernel/userspace split; and a namespace as a language feature: a resolver is a core parameter, paths are a declared type compared structurally, and the namespace literal compiles to a resolver.

Contributions.

- A language whose staging boundary is forced rather than chosen, and whose checker enforces it before the expander runs: the positions that require data — equality, quotation, a namespace key, a path — refuse functions, so no accepted program can ask for an operation the calculus cannot perform (§9).
- Object-language declarations that generate an interpreter’s furniture — constructors, case, spine walker, rebuilder, default arms and fuel loop — which makes authoring a semantics at one site a two-line edit rather than a term construction (§7).
- A scry namespace as a language feature: typed paths, a resolver as a core parameter, a namespace literal that compiles to a resolver, and blocking whose soundness rests on the namespace being append-only, with the non-neutrality of fuel across a resume stated rather than hidden (§8).
- An expander that is a fold over the syntax tree, with bijective laws — $expand \circ render = lower$ and $lower \circ lift = id$ — and a published dictionary keyed by structural hash, which is the same key a VM matches on (§10).
- Evidence that the abstraction costs nothing, by equality of terms: six programs compile byte-identically to artifacts built by hand before this language existed, and reproduce their contraction counts (§10).

¹Stated for the λ -calculus and for $\beta\eta$ -distinct normal forms; see also Barendregt [1984, §10.4]. The probes this paper uses run on Scott-encoded data under weak reduction, which is not that setting, so we cite separability for the asymmetry it names and not as a completeness result for the method.

2 Background

[One page, cited and not restated. *SKI* and bracket abstraction; Scott encoding and the continuation orders that are the ABI; the artifact of Davis — *whnfF*, the unquoter *UQ*, the fuel semantics in which fuel k permits k step-attempts, and Proposition 3.1 — and the counts this paper reproduces rather than re-derives. Fix the measure convention here as well, since four rulers get used on the same object and a reader who confuses them will think the paper contradicts itself: *whnfF* is 618 *atoms*, 1,235 *tree nodes*, 348 distinct nodes once sharing is counted, and 2,109 *characters* when printed. Atoms are the unit for every claim in this paper; the others appear only where they are named. Everything a reader needs for §4 and nothing more. The separation of credit belongs here: the first paper establishes that a self-interpreter exists and what it costs; this one asks what a language owes it.]

3 Lineage

SKIjack is a language entirely based on the *SKI* combinator calculus, and in particular on improving the expressiveness of *SKI* reasoning by the introduction of a standard machinery in the form of generated supercombinators. We survey the historical and technical lineage of pure combinator computation that informs SKIjack’s design. SKIjack is far from the first language or machine concept to lean into pure combinator computing. The present paper leans on five classic results from Corrado Böhm’s work:

1. *CUCH*. Böhm and Gross [1966] introduced the CUCH (CURry-CHurch), a surface for writing programs whose meaning is given by combinators. The *SKI* calculus (including SKIjack) operates under a more severe constraint: while CUCH’s designers were able to place its affordances such as quotation wherever they pleased, pure *SKI* calculus cannot quote expressions (Proposition 3.1 of Davis).
2. *Separability*. TODO
3. *Term-algebra schema*. A type declaration generates its constructors and its case form by the schema Böhm and Berarducci [1985] gave for term algebras, in the Scott variant [Scott, 1962, Jansen, 2013] rather than the Church one: an interpreter’s step function wants case analysis in place of a fold which fixes the declaration order as the continuation order.
4. *First self-compiling compiler*. The first compiler written in its own language is recognized as Böhm [1954], his 1951 dissertation at ETH Zürich [Knuth and Pardo, 1980]. In the case of SKIjack, the compiler carries out quotation at compile time.
5. *Self-interpreter with a normal form*. Finally, Berarducci and Böhm [1993] gave a self-interpreter of the λ -calculus having a normal form.

Bracket abstraction permits the procedural compilation of a high-level applicative language to combinators [?]; there is a related abstraction algorithm [?]. ? introduced the notion of a supercombinator, the closed abstraction compiled as one unit. This program effloresced in the “*SKI Machine*” (SKIM), a graph-reduction machine built on the “Turner set” of combinators, namely, *S*, *K*, *I*, *B*, *C*, *Y*, and *P* [??] and its successor [??]. (A standard account of the whole pipeline has been given by ?.) The SKIM tradition operated with a fixed set of recognized combinators executed natively in microcode, an accelerated process comparable to the “jet” approach we will adopt in this work. Much of this work was concerned with the cost of any particular abstraction, in combinators per source symbol [??], and we report atom counts in line with that tradition.

In the preceding work, the interpreter lived either in a machine (Turner) or in microcode (SKIM), with the reduction engine as an external given. The object introduced by Davis is a 618-atom self-interpreter which exists as a term within the language it interprets. Previous supercombinator work has had no reason to write such an artifact; its self-application question was the compiler bootstrapping in its own source language, not the reducer reducing itself, and so the tower, the per-level cost, and the encoding boundary never arose there.

However, pursuant to the introduction of a self-interpreter as a term within the language, new considerations arise. Staging becomes a design problem: Turner’s SASL could not quote either, but because no SASL program was trying to run another SASL program under a fuel budget, there was no obligation to address the lack. Declaring an object language becomes a language feature, since an interpreter’s constructors, case form, spine walker, rebuilder and fuel loop can be generated from the declaration rather than written. Finally, the native compiler owes its output a strict discipline: expansion is bijective, terms lift back to named source, and the dictionary is published and keyed by structural hash. This discipline becomes crucial because the emitted term is simultaneously the object of study and the key a VM matches on, whereas nobody expected to read the output of a SASL compiler.

This last merits an aside. Our VM is capable of identifying supercombinators to arbitrary size and reducing them instantly against a table of known outcomes in native code (what we call a “jet”, short for “jet-accelerated code”). Mechanically, such a procedure corresponds to Turner’s extended combinator set and SKIM’s microcode. However, our theorem does not licence arbitrary native code replacement; the interpreter jet performs a structural recognition that no term can perform for itself and thus is not an optimization of an operation the language has. The operational consequence of this prohibits a jet from quoting on behalf of a running term. The VM is addressed briefly in §11.

4 The interpreter

The `whnfF` function forms the centerpiece of the interpreter, responsible for reducing terms to weak head normal form. It orchestrates the evaluation strategy for the language, ensuring that terms are reduced correctly according to the operational semantics. It also suggests the language features that are necessary to support such an evaluation strategy, which will emerge in SKIjack as primitives. There is also an unquoter function `UQ`, responsible for converting weak head normal forms back into their original syntactic representation. This is necessary for maintaining the correspondence between the evaluated terms and their original syntactic forms, ensuring that the interpreter’s output can be related back to the source code. In Davis, we simply named the features by their operational significance as a shorthand; here they become language elements in their own right.

Table 1 gives some typed forms with their defining equations and with closed $\{S, K, I\}$ terms emitted by the expander (i.e., compiled output). Three of the six reduce to the same term, and two more are a second same term. This results because a Scott datum is its own case analysis, so the first constructor of any two-constructor type discards the remaining continuation and is K , whatever the type was called, and a one-field second constructor applies one continuation to one field and is the same eight atoms whether the field is a numeral or a term. At the level of terms the type distinctions are erased. SKIjack maintains the type discipline at compile time (§9). A checker may reject a program that offers a numeral where a list is wanted, but nothing downstream of the expander could tell the two apart, because downstream of the expander they

are one object. (It is also why the dictionary that names these expansions is keyed by the hash of a term and not by a name.)

Table 1: Some significant forms, their definitions, and the terms they compile to. The first constructor of each pair is the same single atom and the one-field second constructors are the same eight; only **cons**, which carries two fields, is distinct.

form	definition	expansion	atoms
nil	$\lambda n.\lambda c.n$	K	1
cons <i>h t</i>	$\lambda n.\lambda c.cht$	S (K (S (K K))) (S (S (K S) (S (K K) (S (K S) (S (K (S I)) K)))) (K K))	22
nothing	$\lambda n.\lambda j.n$	K	1
just <i>x</i>	$\lambda n.\lambda j.jx$	S (K K) (S (K (S I)) K)	8
zero	$\lambda z.\lambda s.z$	K	1
suc <i>n</i>	$\lambda z.\lambda s.sn$	S (K K) (S (K (S I)) K)	8

The metacircular evaluator uses these object terms to represent the basic constructs of the language within itself. Next, **whnfF** itself is composed of 618 atoms built from a few pieces. Table 2 inventories **whnfF**'s parts, smallest first, with the term each part compiles to. By reading the lower block from the bottom upwards, the interpreter assembles itself out of the upper one.

1. **hd**, the head extractor;
2. **tl**, the tail extractor;
3. **resI**, the result *I* arm;
4. **resK**, the result *K* arm;
5. **Y**, the fixed-point combinator;
6. **resS**, the result *S* arm;
7. **pair**, the pair constructor;
8. **App**, the application constructor;
9. **UQ**, the unquoter;
10. **rb1**, the first rebuilder arm;
11. **spApp**, the spine application handler;
12. **rb**, the rebuilder;
13. **stepI1**, the first step *I* arm;
14. **stepK2**, the second step *K* arm;
15. **stepI**, the step *I* arm;
16. **stepK1**, the first step *K* arm;
17. **stepK**, the step *K* arm;
18. **sp**, the spine walker;
19. **stepsS3**, the third step *S* arm;
20. **stepsS2**, the second step *S* arm;
21. **stepsS1**, the first step *S* arm;
22. **stepS**, the step *S* arm;
23. **step**, the step walker from the spine to the head leaf;
24. **loop1**, the first loop arm; and
25. **whnfF**, the whole interpreter.

Note that the expansions of **rb**, **UQ**, **sp**, and **whnfF** begin with the fourteen atoms of *Y*. The fixpoint tie is a prefix of the emitted term, and which arms recurse can be read off the object itself. This is perhaps the one piece of the surface's structure that survives bracket abstraction in a form a reader can observe for himself.

5 The language

The **whnfF** function of Davis introduced a number of independent operational terms within its full expression, and these suggest language features such as its loop, its comparison and dispatch, and its atoms (**suc**, **zero**, **nil**, **just**, etc.). These features were composed into larger machine terms, which are similar to the supercombinators of ?.

Table 2: Every named part of the **whnfF** interpreter, ordered by size. Above the rule, the closed $\{S, K, I\}$ term in full; below it, the same term named against the parts above by *lift*, which renames exact structural matches as a macro.

part	atoms	expansion
hd	4	$S\ I\ (K\ K)$
tl	5	$S\ I\ (K\ (K\ I))$
resI	11	$S\ (K\ K)\ (S\ (K\ K)\ (S\ (K\ (S\ I))\ K))$
resK	13	$S\ (K\ K)\ (S\ (K\ (S\ (K\ K)))\ (S\ (K\ (S\ I))\ K))$
Y	14	$S\ (K\ (S\ I\ I))\ (S\ (S\ (K\ S)\ K)\ (K\ (S\ I\ I)))$
resS	15	$S\ (K\ (S\ (K\ K)))\ (S\ (K\ (S\ (K\ K)))\ (S\ (K\ (S\ I))\ K))$
pair	17	$S\ (S\ (K\ S)\ (S\ (K\ K)\ (S\ (K\ S)\ (S\ (K\ (S\ I))\ K))))\ (K\ K)$
App	32	$S\ (K\ (S\ (K\ K)))\ (S\ (K\ (S\ (K\ K)))\ (S\ (K\ (S\ (K\ K)))\ (S\ (S\ (K\ S)\ (S\ (K\ K)\ (S\ (K\ S)\ (S\ (K\ (S\ I))\ K))))\ (K\ K)))$
UQ	43	$S\ (K\ (S\ I\ I))\ (S\ (S\ (K\ S)\ K)\ (K\ (S\ I\ I)))\ (S\ (K\ (S\ (S\ (S\ (S\ I\ (K\ S))\ (K\ K))\ (K\ I))))\ (S\ (K\ K)\ (S\ (S\ (K\ S)\ (S\ (K\ (S\ (K\ S)))\ (S\ (K\ K))))\ K)))$
rb1	44	$S\ (S\ (K\ S)\ (S\ (K\ K)\ (S\ (K\ S)\ K)))\ (K\ (S\ (K\ (S\ (K\ K)))\ (S\ (K\ (S\ (K\ K)))\ (S\ (K\ (S\ (K\ K)))\ (S\ (S\ (K\ S)\ (S\ (K\ K)\ (S\ (K\ S)\ (S\ (K\ (S\ I))\ K))))\ (K\ K))))))$
spApp	48	$S\ (S\ (K\ S)\ (S\ (K\ K)\ (S\ (K\ S)\ (S\ (K\ (S\ (K\ S)))\ (S\ (K\ K))))\ (K\ (S\ (K\ K)\ (S\ (K\ (S\ (S\ (K\ (S\ (K\ K)))\ (S\ (S\ (K\ S)\ (S\ (K\ K)\ (S\ (K\ S)\ (S\ (K\ (S\ I))\ K))))\ (K\ K))))\ K)))$
rb	74	$S\ (K\ (S\ I\ I))\ (S\ (S\ (K\ S)\ K)\ (K\ (S\ I\ I)))\ (S\ (K\ (S\ (S\ (K\ S)\ (S\ (K\ (S\ I))\ K))))\ (S\ (K\ (S\ (K\ K)))\ (S\ (S\ (K\ S)\ (S\ (K\ S)\ (S\ (K\ S)\ K)))\ (K\ (S\ (K\ K)))\ (S\ (K\ (S\ (K\ K)))\ (S\ (K\ (S\ (K\ K)))\ (S\ (K\ (S\ (K\ K))\ (S\ (K\ S)\ (S\ (K\ K)\ (S\ (K\ S)\ (S\ (K\ (S\ I))\ K))))\ (K\ K))))))\ K)))$
stepI1	86	$S\ (K\ (S\ (K\ just)))\ rb$
stepK2	89	$S\ (K\ K)\ stepI1$
stepI	92	$S\ hd\ (K\ stepI1)$
stepK1	99	$S\ (K\ (S\ hd))\ (S\ (K\ K)\ stepK2)$
stepK	105	$S\ hd\ (K\ stepK1)$
sp	126	$Y\ (S\ (K\ (S\ (S\ (K\ S)\ (S\ (S\ (K\ S)\ (S\ (S\ (K\ S)\ (S\ (S\ (K\ S)\ K)\ (K\ resS)))\ (K\ resK)))\ (K\ resI))))\ (S\ (K\ K)\ spApp))$
stepS3	207	$S\ (K\ (S\ (K\ (S\ (K\ (S\ (K\ Just))))))\ (S\ (K\ (S\ (K\ (S\ (K\ rb))))\ (S\ (S\ (K\ S)\ (S\ (K\ K)\ (S\ (K\ S)\ (S\ (K\ (S\ (K\ App)))\ (K\ App))))\ (K\ App)))$
stepS2	221	$S\ (K\ (S\ (K\ (S\ hd))))\ (S\ (K\ (S\ (K\ K)))\ stepS3)$
stepS1	231	$S\ (K\ (S\ hd))\ (S\ (K\ K)\ stepS2)$
stepS	237	$S\ hd\ (K\ stepS1)$
step	569	$S\ (S\ (S\ (S\ sp\ (K\ K))\ (K\ stepS))\ (K\ stepK))\ (K\ stepI)$
loop1	588	$S\ (K\ (S\ (S\ (K\ S)\ (S\ (K\ K)\ (S\ step\ Just))))\ K$
whnfF	618	$Y\ (S\ (K\ (S\ (S\ (K\ S)\ (S\ (K\ K)\ hd)))\ (S\ (K\ K)\ loop1))$

SKIjack's features are written as a set of object terms in the metacircular evaluator; in essence, it is a macro language with a gently typed surface discipline. They name and highlight the components of **whnfF** and other recurrent supercombinators, augmented with affordances to make the language expressive and composable.

Table 3: The object terms of *SKI* in the metacircular evaluator.

form	definition	expansion	atoms
nil	$\lambda n. \lambda c. n$	K	1
cons $h\ t$	$\lambda n. \lambda c. c\ h\ t$	$S\ (K\ (S\ (K\ K)))\ (S\ (S\ (K\ S)\ (S\ (K\ K)\ (S\ (K\ S)\ (S\ (K\ (S\ I))\ K))))\ (K\ K))$	22
nothing	$\lambda n. \lambda j. n$	K	1
just x	$\lambda n. \lambda j. j\ x$	$S\ (K\ K)\ (S\ (K\ (S\ I))\ K)$	8
zero	$\lambda z. \lambda s. z$	K	1
suc n	$\lambda z. \lambda s. s\ n$	$S\ (K\ K)\ (S\ (K\ (S\ I))\ K)$	8

5.1 Design principles

Since quotation of live terms is not supported, the language relies on the macro-like composition of its object terms rather than runtime evaluation. A hard line between runtime quotation and compile-time quotation is forced by Proposition 3.1 of [Davis](#), which follows from Theorem 3.2 of [Jay and Given-Wilson \[2011\]](#): “No closed term Q over $\{S, K, I\}$ satisfies $Q\ M \rightarrow \langle M \rangle$ for every closed M .” Everything on the encoded side of that line is closed under the calculus, including running encoded terms, comparing them, and building cores. Everything that would turn a live value back into data is unavailable and must be staged via the virtual machine. One design problem is to make that line visible and comfortable. Another difference from traditional macro languages is that weak SKI reduction is call-by-name: arguments are never forced early, **K** **I** **Ω** is **I**, and no form needs thinking.

The basic components of SKIjack are thus: a macro layer; Scott encoding; recursion through **Y**; interpreters, the unquoter, and equality on encoded terms like paths; the behavioral verification discipline; and the existence of a runtime which can jet the compiled code for more efficient execution.

SKIjack supplies one grammar with two lexicons (ASCII-only and Unicode), with a bijective token table checked at import, and the round-trip and lift laws as tests. Given agent-driven development workflows, it is pragmatic to maintain both depending on the aesthetic and practical considerations of the development environment. Listing 1 and Listing 2 show the same simple example.

```

seg  ≡ Nat | Two | Three
path ≡ Nil | Cons seg path
resolve ≡ []/nat/two ↦ <>I, /nat/three ↦ <>[]K
answer ≡ wfN resolve ⊢ <::/nat/⟩10three

```

Listing 1: A simple example of SKIjack code under the Unicode lexicon.

```

seg  === Nat | Two | Three
path === Nil | Cons seg path
resolve := ns{/nat/two => <I>, /nat/three => <K>}
answer := wfN resolve |- <?^/nat/three>@10

```

Listing 2: A simple example of SKIjack code under the ASCII lexicon.

5.2 Representation ABI

5.3 Kernel forms

5.4 Supercombinators vs. macros

5.5 The standard subject

5.6 The quotation line

```
term === S | K | I | App term term
maybe === Nothing | Just term
whnfF := {
  step m = sp m nil stepS stepK stepI
  loop1 f m n2 = step m (Just m) (f n2)
  loop n m = n Nothing (loop1 loop m)
}
resS acc c0 c1 c2 = c0 acc
resK acc c0 c1 c2 = c1 acc
resI acc c0 c1 c2 = c2 acc
spApp f acc t u = f t (cons u acc)
sp m acc = m (resS acc) (resK acc) (resI acc) (spApp sp acc)
rb1 f h x xs = f (App h x) xs
rb h args = args h (rb1 rb h)
stepI1 x rest = Just (rb x rest)
stepI args = args Nothing stepI1
stepK2 x y rest = Just (rb x rest)
stepK1 x r = r Nothing (stepK2 x)
stepK args = args Nothing stepK1
stepS3 x y z rest = Just (rb (App (App x z) (App y z)) rest)
stepS2 x y r2 = r2 Nothing (stepS3 x y)
stepS1 x r = r Nothing (stepS2 x)
stepS args = args Nothing stepS1
```

Listing 3: The `whnfF` function defined in SKIjack. Pleasingly, this program produces the identical *SKI* artifact as Appendix B of [Davis](#) despite SKIjack not existing at the time of composition.

SKIjack is not self-bootstrapping: in any case, self-compilation would require a parser which mapped arbitrary ASCII input to the internal representation of SKIjack terms. While not impossible, the operational overhead does not seem to merit the additional effort or reveal new premises for *SKI* as a tool. Furthermore, quotation of live terms is not supported, which limits the ability to perform self-representation within the language itself (no runtime `eval`). The language is Turing-complete but certain functions are transferred to the VM, such as scaffolding the initial evaluation and preserving the state and memory arena.

6 Execution model

7 Interpreters as declarations

[The payoff section, and where the pair of papers is joined. The declaration form and what it generates; then the one-site extension, with `wf5Abs` and `wf50mg` differing at a single arm and compiling to 768 and 771 atoms, byte-identical to the artifacts; the T3 observability table reproduced. The sentence this section exists to support: the first paper's move — author a semantics at one site — is here a declaration form, and its cost over the hand construction is zero atoms.]

8 The namespace

[Typed paths as a declared type rather than arbitrary terms; the resolver as a core parameter; the namespace literal and what it compiles to; blocking, the resume loop, and why re-running from scratch is what makes it sound under an append-only namespace; the value-lookup example. State the two honest wrinkles in the text and not a footnote: the monotonicity hypothesis, and that fuel is not neutral across a resume.]

9 Boundary soundness

[Recommended, and the paper's own theorem rather than a borrowed one. Define the fragment; state that an accepted program never places a function where data is required, so the operations the calculus cannot perform are refused before they can be reached; prove by induction on the checker. Add erasure as a corollary — checking can reject but never rewrites, so the emitted term is the same whether it ran or not — which is what keeps the laws of §10 intact. If this section does not survive, say plainly in §1 that the enforcement is tested and not proved.]

10 Implementation and evaluation

[The expander as a fold, its passes, and the dictionary. The four laws as executable tests. The byte-identity table — `whnfF` 618, `wf5Abs` 768, `wf50mg` 771, `wfQ` 950, `wfN` 1,066, `EQ5` 240, with `T0` at 340 — and, in the same table, the distinction that answers the circularity objection before it is raised: the generated forms were designed to reproduce the artifact, whereas the long version, written out as ordinary equations with generation switched off, reaches the same term without that help. The corpus and the test count. One sentence stating that no performance claim is made here and why: the reference host is an oracle, not an implementation, and the VM is discussed only briefly.]

11 The virtual machine

12 Limitations, and what is not claimed

[Self-hosting, declined, with the reason stated precisely: the expander's input is a syntax tree, so it is separable from the parser, and what is forgone is not a demonstration but the Futamura projections inside the calculus. The type discipline stops at Stage A. There is no standard subject yet, so a

level-1 library name is inlined rather than shared. And the precision this section must carry: the language has evaluation of encoded terms, including terms built at run time, and lacks reification of live values — the missing operation is *reify*, not *eval*. The paragraph currently at the end of §5 belongs here.]

References

- Henk P. Barendregt. *The Lambda Calculus: Its Syntax and Semantics*, volume 103 of *Studies in Logic and the Foundations of Mathematics*. North-Holland, Amsterdam, revised edition, 1984.
- Alessandro Berarducci and Corrado Böhm. A self-interpreter of lambda calculus having a normal form. In *Computer Science Logic (CSL '92)*, volume 702 of *Lecture Notes in Computer Science*, pages 85–99. Springer, 1993.
- Corrado Böhm. Calculatrices digitales: du déchiffrement de formules logico-mathématiques par la machine même dans la conception du programme. *Annali di Matematica Pura ed Applicata*, 37:175–217, 1954. Series 4. Originally the author’s doctoral dissertation, ETH Zürich, 1951.
- Corrado Böhm. Alcune proprietà delle forme β - η -normali nel λ -K-calcolo. Technical Report 696, Istituto per le Applicazioni del Calcolo, Rome, 1968.
- Corrado Böhm and Alessandro Berarducci. Automatic synthesis of typed λ -programs on term algebras. *Theoretical Computer Science*, 39:135–154, 1985.
- Corrado Böhm and Wolf Gross. Introduction to the CUCH. In Eduardo R. Caianiello, editor, *Automata Theory*, pages 35–65. Academic Press, New York, 1966.
- N. E. Davis. A self-interpreter for *ski* written in *ski*: Extensional semantics for new symbols. *Journal of Functional Programming*. In review.
- Jan Martin Jansen. Programming in the λ -calculus: From Church to Scott and back. In *The Beauty of Functional Code*, volume 8106 of *Lecture Notes in Computer Science*, pages 168–180. Springer, 2013.
- Barry Jay and Thomas Given-Wilson. A combinatory account of internal structure. *Journal of Symbolic Logic*, 76(3):807–826, 2011.
- Donald E. Knuth and Luis Trabb Pardo. The early development of programming languages. In N. Metropolis, J. Howlett, and Gian-Carlo Rota, editors, *A History of Computing in the Twentieth Century*, pages 197–273. Academic Press, New York, 1980.
- Dana S. Scott. A system of functional abstraction. Lectures delivered at the University of California, Berkeley, 1962/63; unpublished manuscript, Stanford University, 1963, 1962.

A The artifact

[The parts below the rule of Table 2, in full, and `whnfF` itself at 618 atoms and 2,109 characters. The first paper prints the same term in its Appendix B; if space is short, cite it there and print only the parts this paper names, since the claim being supported is byte-identity and a reader who wants to check it needs the term from one of the two.]